NCAA Crystal Ball

Milestone 17

Milestone 17

We are going to use cross-validation to tune to cutoff for what probability is needed for our model for a team to be considered a "contender". We are gonna use our original model where it predicts whether a team wins the NCAA national championship based on AdjEM.

Load data

```
from pathlib import Path
import pandas as pd

paths = [
    Path("data/processed/champion_checklist.csv"),
    Path("../data/processed/champion_checklist.csv")
]

data_path = next((p for p in paths if p.exists()), None)

if data_path is None:
    raise FileNotFoundError("Could not locate champion_checklist.csv")

data = pd.read_csv(data_path)

print(f"Loaded data from: {data_path}")

# Keep all Division I teams, not just tournament teams
teams = data[
    data["is_d1"].astype(str).str.strip().str.lower().isin(
```

```
        ["true", "t", "1"]
    )
].copy()

# Keep observations with the inputs we need
teams = teams.dropna(subset=["season", "adj_em", "champion"])

# Scale AdjEM the same way as Milestone 15
teams["adj_em_5"] = (teams["adj_em"] - 20) / 5

# Keep later seasons separate for testing
train = teams[teams["season"] <= 2021].copy()
test = teams[teams["season"] >= 2022].copy()

print(f"Training observations: {len(train)}")
print(f"Training seasons: {train['season'].nunique()}")
print(f"Training champions: {train['champion'].sum()}")

print(f"\nTest observations: {len(test)}")
print(f"Test seasons: {test['season'].nunique()}")
print(f"Test champions: {test['champion'].sum()}")
```

```
Loaded data from: ../data/processed/champion_checklist.csv
Training observations: 5509
Training seasons: 16
Training champions: 16

Test observations: 1802
Test seasons: 5
Test champions: 5
```

5-Fold Cross-Validation

Instead of randomly splitting teams, lets group our teams by season so those teams competing in the same year are in the same fold

For each fold, train the model on the four groups and predict the probability.

We are gonna keep 2022-26 seasons seperate for testing.

```python
import numpy as np
import statsmodels.api as sm
import statsmodels.formula.api as smf
from sklearn.model_selection import GroupKFold

# Set up 5-fold cross-validation by season
cv = GroupKFold(n_splits=5)

# Store the cross-validation probabilities
train["cv_probability"] = np.nan

for fold, (train_idx, valid_idx) in enumerate(
    cv.split(train, groups=train["season"]), start=1
):

    # Divide the training and validation data
    fold_train = train.iloc[train_idx]
    fold_valid = train.iloc[valid_idx]

    # Fit our logistic regression using AdjEM
    model = smf.glm(
        "champion ~ adj_em_5",
        data=fold_train,
        family=sm.families.Binomial()
    ).fit()

    # Predict probabilities for the validation teams
    train.loc[fold_valid.index, "cv_probability"] = model.predict(fold_valid)

    print(
        f"Fold {fold}: "
        f"{len(fold_valid)} teams, "
        f"{fold_valid['season'].nunique()} seasons, "
        f"{int(fold_valid['champion'].sum())} champions"
    )

# Check that every training team received a prediction
assert train["cv_probability"].notna().all()

print("\nCross-validation complete!")
print(f"Predictions generated: {len(train)}")
```

```
Fold 1: 1366 teams, 4 seasons, 4 champions
Fold 2: 1035 teams, 3 seasons, 3 champions
Fold 3: 1035 teams, 3 seasons, 3 champions
Fold 4: 1037 teams, 3 seasons, 3 champions
Fold 5: 1036 teams, 3 seasons, 3 champions

Cross-validation complete!
Predictions generated: 5509
```

Finding our cutoff

We are gonna test a bunch of different values and measuring accuracy, precision of how close, recall and F1 score. We are using F1 score because our dataset has one champ per season. The goal here is to find the cutoff that finds the actual champions without incorrectly choosing too many other teams

```python
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Different cutoff probabilities to test
cutoffs = [0.001, 0.002, 0.005, 0.01, 0.02,
           0.03, 0.05, 0.10, 0.15, 0.20]

results = []

for cutoff in cutoffs:

    # Classify teams using the cutoff
    predictions = (train["cv_probability"] >= cutoff).astype(int)

    # Evaluate the classifications
    results.append({
        "Cutoff": cutoff,
        "Accuracy": accuracy_score(train["champion"], predictions),
        "Precision": precision_score(
            train["champion"], predictions, zero_division=0
        ),
        "Recall": recall_score(
            train["champion"], predictions, zero_division=0
        ),
        "F1": f1_score(
            train["champion"], predictions, zero_division=0
        ),
```

```
        "Teams Flagged": int(predictions.sum())
    })

# Compare our cutoff probabilities
results_df = pd.DataFrame(results)

print("Cross-validation results:")
print(results_df.round(4).to_string(index=False))

# Select the cutoff with the highest F1 score
best = results_df.sort_values(
    ["F1", "Recall", "Cutoff"],
    ascending=[False, False, True]
).iloc[0]

print(f"\nBest cutoff probability: {best['Cutoff']:.2%}")
print(f"F1 score: {best['F1']:.4f}")
print(f"Precision: {best['Precision']:.4f}")
print(f"Recall: {best['Recall']:.4f}")
```

```
Cross-validation results:
 Cutoff  Accuracy  Precision  Recall     F1  Teams Flagged
  0.001    0.8813     0.0239  1.0000 0.0466            670
  0.002    0.9134     0.0325  1.0000 0.0629            493
  0.005    0.9455     0.0478  0.9375 0.0909            314
  0.010    0.9617     0.0628  0.8750 0.1172            223
  0.020    0.9764     0.0929  0.8125 0.1667            140
  0.030    0.9820     0.1193  0.8125 0.2080            109
  0.050    0.9880     0.1528  0.6875 0.2500             72
  0.100    0.9935     0.2059  0.4375 0.2800             34
  0.150    0.9949     0.2692  0.4375 0.3333             26
  0.200    0.9955     0.2353  0.2500 0.2424             17

Best cutoff probability: 15.00%
F1 score: 0.3333
Precision: 0.2692
Recall: 0.4375
```

So we compared 10 different cutoff values from 0.1% to 20%

The cutoff with the highest F1 score was 15%, with a socre of 0.3333, precision of 26.92%, and recall of 43.75%

Interpretations

Pecision: 26.62% of teams our model saw as potential champions actually became champs.

Recall: Our model identfied 43.75% of all the national champions

F1 Score: This shows that our current model with one predictor is limited to balancing its ability to flag and contenders and see actual champions

At this value, the model identified 7 of the 16 champions, while flagging 26 teams total. This shows that our model can identify some championship contenders, but it still misses several actual champions.

Note

This is our model with one predictor. As we add other predictors for our model we plan to see these validations get a lot better. And penalized regression will also help with adding multiple predictors here